

Week 4- Sorting Algorithms I

Condensed Study Notes

Generated: 2026-05-25 10:49 UTC

Week 4- Sorting Algorithms I

Sorting algorithms arrange elements in a list into a specific order (e.g., ascending or descending).

This section introduces fundamental sorting algorithms.

Key terms:

- **List:** A collection of elements.
- **Sorted:** Elements arranged in a defined order.
- **Unsorted:** Elements not arranged in a defined order.

Overview

Sorting is the process of rearranging a sequence of objects (elements) into a specific logical order. This involves defining an ordering rule and applying it to reorder the elements of an input, typically an array.

Sorting is a fundamental operation in computing, often used as an intermediate step in many programs.

Categories of sorting

Sorting algorithms are categorized based on whether the data to be sorted can fit into the computer's main memory.

- **Internal sorting:** This occurs when all elements to be sorted reside in memory simultaneously.
- **External sorting:** This is used when the dataset is too large to fit into memory all at once. It involves sorting in phases, often using external storage (like a hard drive) for large data and intermediate sorted files, and then combining the results. This is essential for massive datasets.

Some sorting terminologies

Key terms related to sorting algorithms:

- **In-place sorting algorithm:** Sorts a list using minimal auxiliary storage, typically requiring only a constant amount of extra memory for variables. It rearranges elements by swapping or replacing them directly within the input list. Algorithms that are not in-place are called out-of-place.
- **Stable sorting algorithm:** Preserves the original relative order of elements that have identical values. If two equal elements are in a specific order in the input, they will remain in that same order in the sorted output.
- **Sorting algorithm:** A method to arrange list elements into a specific order, most commonly numerical or lexicographical (alphabetical). This order can be non-decreasing (ascending) or non-increasing

(descending).

- **Sorting problem:** A computational task that is resolved by using a sorting algorithm.

Common sorting algorithms

We will explore six common sorting algorithms, starting with three elementary ones:

- **Selection Sort**
- **Bubble Sort**
- **Insertion Sort**

Selection sort

Selection sort is a brute-force sorting algorithm. In each pass, it finds the smallest element from the unsorted portion of the list and moves it to the beginning of that unsorted portion.

Imagine you have a pile of unsorted cards and you want to sort them by number. Selection sort is like picking out the smallest card from the whole pile and placing it down as the first card in your sorted stack, then repeating this process with the remaining cards.

Selection sort algorithm

Selection sort works by repeatedly finding the minimum element from the unsorted part of the list and placing it at the beginning of the sorted part.

It's like picking out the smallest item from a pile of unsorted objects and setting it aside as the first item in your sorted collection, then repeating the process with the remaining pile.

Key steps:

- Identify the smallest element in the unsorted portion of the list.
- Swap this smallest element with the first element of the unsorted portion.
- The first element is now considered sorted, and the process continues with the rest of the list.

The selection sort is used when

Selection sort is a good choice in specific scenarios:

- When every element in the list must be examined.
- For sorting very small lists.
- When the cost of swapping elements is not a concern.
- In situations where writing to memory is expensive, such as with flash memory, because selection sort performs a predictable number of writes (swaps), proportional to the list size ($\mathcal{O}(n)$).

Bubble sort

Bubble sort is another brute-force sorting method.

It works by comparing adjacent elements in a list. If they are in the wrong order, they are swapped.

This process is repeated, causing the largest elements to 'bubble up' to their correct positions at the end of the list.

Bubble sort algorithm

Bubble sort arranges a list by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. This process causes larger elements to 'bubble up' to their correct positions at the end of the list.

The algorithm repeatedly passes through the list. In each pass, it compares each pair of adjacent items. If the pair is in the wrong order, they are swapped. This continues until no swaps are needed in a pass, indicating the list is sorted.

The bubble sort is used when

Bubble sort is chosen when:

- Simplicity of code is prioritized.
- The complexity of the sorting process is not a significant concern.

Insertion sort

Insertion sort places an unsorted element into its correct position within the already sorted portion of the list in each step.

It uses a decrease-and-conquer strategy, specifically 'decrease-by-one'. This means it assumes a smaller part of the list is already sorted and uses that solution to find the right spot for the next unsorted element.

Think of it like building a hand of playing cards. You pick up one card at a time and insert it into its correct sorted position within the cards you already hold.

Insertion sort algorithm

Insertion sort builds a sorted list by taking unsorted elements one by one and inserting them into their correct positions within the already sorted part of the list.

Think of sorting a hand of playing cards. You pick up one card at a time and place it where it belongs among the cards already in your hand.

Key points:

- It processes the list sequentially.
- Each element is compared with elements in the sorted sublist and shifted until its correct position is found.
- This method is effective for small datasets or nearly sorted lists.

Insertion Sort is used when

Insertion sort is particularly effective in two scenarios:

- **Small datasets:** When the entire array contains a small number of elements.
- **Nearly sorted data:** When only a few elements remain to be placed in their correct positions within an otherwise sorted list.

Summary of complexities of the elementary sorting algorithms

This section summarizes the performance characteristics of Bubble Sort, Selection Sort, and Insertion Sort.

Time Complexity (how runtime scales with input size 'n'):

- **Overall:** All three algorithms have an overall time complexity of $O(n^2)$.
- **Best Case:** Bubble Sort and Insertion Sort are $O(n)$ if the list is already sorted, while Selection Sort remains $O(n^2)$.
- **Average & Worst Case:** All three algorithms have an average and worst-case time complexity of $O(n^2)$.

Swaps/Movements (operations to rearrange elements):

- **Best Case:** All algorithms perform minimal swaps/movements in the best case ($O(1)$ for Bubble/Insertion, $O(n)$ for Selection).
- **Worst Case:** Bubble Sort and Insertion Sort require $O(n^2)$ swaps, while Selection Sort requires $O(n)$.

Auxiliary Space: All three algorithms use $O(1)$ auxiliary space, meaning they sort in-place without needing significant extra memory.

Algorithmic Paradigm:

- Bubble Sort and Selection Sort are considered 'Brute Force' methods.
- Insertion Sort uses a 'Decrease and Conquer' approach.

Sorting Method:

- Bubble Sort and Selection Sort rely on 'Swapping' elements.
- Insertion Sort uses 'Shifting' elements.

Sorting in Place: All three algorithms perform sorting in-place.

Stability:

- Bubble Sort and Insertion Sort are 'Stable' sorting algorithms.

- Selection Sort is 'Not Stable'.